Spike técnico ST-01

Canales de captura y modelo canónico de notificación

Versión	0.2
Fecha	17 de octubre de 2026
Estado	recomendación técnica provisional; no es una decisión arquitectónica final
Base documental	PRD CNF-PRD-001 v0.3; SRS CNF-SRS-001 v0.3; S-0.1 (escenario simulado).

S-0.1 es un escenario simulado, no resultados de entrevistas realizadas. Sus cifras son ficticias y no validan segmento, preferencias ni alcance del MVP.

Producto y requisitos · Documento de trabajo

1. Resumen ejecutivo

El spike reduce dos riesgos distintos: **cómo recibir mensajes de pago** y **cómo impedir que la lógica de dominio dependa del canal de entrada**. Mantener el dominio neutral respecto al canal y posponer la selección final hasta realizar pruebas con emisores y dispositivos autorizados. S-0.1 es un escenario simulado, no resultados de entrevistas realizadas; por tanto, no justifica prioridades ni preferencias de canal.

- **Lectura de Gmail mediante OAuth:** no se recomienda como primera ruta del MVP. El scope `gmail.readonly` está clasificado por Google como restringido; cuando datos de esos scopes se almacenan en servidores o se transmiten, Google exige una evaluación de seguridad, además de verificación OAuth [3]. No es imposible, pero eleva el esfuerzo de autorización y cumplimiento.
- **Reenvío de correo con filtro:** candidato más acotado para una prueba si los emisores prioritarios notifican por correo. Gmail permite filtrar qué mensajes reenvía, pero la persona debe configurar y verificar la dirección de destino; el contenido elegido se transfiere al servicio [4]. Esto no elimina responsabilidades de privacidad o retención.
- **Listener Android:** cubre notificaciones publicadas por apps, no solo correos. Android define `NotificationListenerService` para recibir eventos de notificaciones y requiere declararlo con `BIND_NOTIFICATION_LISTENER_SERVICE`; la acción del sistema abre la pantalla de ajustes de acceso del listener [1][2]. Requiere construir y mantener un componente Android, y probar dispositivos, versiones, reinicios y revocaciones.
- **Vertical slice:** 12/12 pruebas automatizadas pasan. Con fixtures inventados se normaliza un pago, se concilia solo ante una coincidencia única, se rechazan falsas confirmaciones y se evita duplicar el mismo pago cuando se observa por correo y Android.

Resultado de esta iteración: el prototipo prueba un contrato común con envelopes sintéticos; no integra Gmail, Android ni formatos de emisores reales. La selección de canal sigue abierta en TBD-1 y debe cerrarse mediante un ADR tras pruebas con muestras autorizadas. El spike técnico no valida la dirección de producto, el segmento ni el alcance del MVP; estos siguen pendientes de validación real. Las entrevistas no bloquean este spike técnico, pero sí se requieren antes de baselinar producto o iniciar piloto.

2. Objetivo, alcance y restricciones

2.1 Objetivo

Verificar que un evento capturado desde distintos canales pueda transformarse a un modelo común, procesarse de forma idempotente y asociarse conservadoramente a un cobro esperado, manteniendo aislamiento por negocio.

2.2 Incluido

1. Comparación inicial de Gmail API, reenvío filtrado y notificaciones Android.
2. Definición del contrato de captura y del pago normalizado.
3. Separación entre idempotencia de entrega y deduplicación de identidad de pago.
4. Reglas sintéticas para emparejar un pago con un cobro esperado.
5. Pruebas de aceptación automatizadas sobre casos felices y negativos.

2.3 Fuera de alcance

No se implementan OAuth, servidor receptor de correo, aplicación Android, APIs de bancos, notificaciones reales, almacenamiento productivo cifrado, usuarios autenticados ni despliegue. El parser reconoce exclusivamente fixtures inventados; **no representa formatos reales de Nequi, Daviplata u otros emisores**. El prototipo está escrito en Python y se considera una especificación ejecutable; no es el backend de producción. Su port a NestJS/TypeScript está pendiente.

3. Comparación de canales

Alternativa	Cobertura potencial	Permisos y fricción	Riesgos/limitaciones	Posición para el MVP
Gmail API con `gmail.readonly`	Mensajes de pago recibidos en una cuenta Gmail	OAuth sobre mensajes y configuración; scope restringido; verificación y evaluación de seguridad al almacenar/transmitir datos de ese scope [3]	Acceso al buzón más amplio que un filtro de emisor; una ruta técnica compleja para iniciar	No usar como primera ruta; reevaluar solo si el valor compensa el costo de autorización y seguridad
Reenvío Gmail filtrado a una dirección de ingestión	Correos de emisores que lleguen al buzón configurado	La persona dueña configura filtro y verifica dirección; el contenido seleccionado se reenvía [4]	No recibe notificaciones que solo existan como push; configuración por negocio; correo transferido requiere protección, minimización y retención	Candidato a probar; prioridad depende de cobertura real y validación con personas usuarias
Android `NotificationListenerService`	Notificaciones publicadas por apps Android compatibles	Se necesita app Android, servicio declarado con `BIND_NOTIFICATION_LISTENER_SERVICE` y acceso habilitado desde ajustes [1][2]	Permiso a nivel de notificaciones; variabilidad por versión/fabricante; perfil de trabajo y dispositivos de baja RAM pueden limitarlo [1]	Alternativa a comparar en dispositivo de prueba; no incluir sin confirmar que el incremento de cobertura compensa la fricción

3.1 Hallazgos documentales

- Android describe el listener como receptor de eventos cuando una notificación se publica, se retira o cambia de ranking. La implementación debe esperar a `onListenerConnected()`; hay limitaciones documentadas en ciertos dispositivos de baja RAM y perfiles de trabajo [1].
- Android expone `Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS` para mostrar la configuración del acceso a listeners [2]. Por tanto, la concesión no se debe modelar como automática: la experiencia requiere una acción explícita de la persona usuaria en el dispositivo.
- Google clasifica `gmail.readonly` como un scope restringido. La documentación indica verificación OAuth y, si los datos restringidos se almacenan o transmiten, evaluación de seguridad [3].
- Gmail permite reenviar mensajes que coinciden con filtros y exige verificar la dirección de reenvío antes de habilitarla [4]. El filtro limita qué mensajes se envían, pero no elimina el tratamiento de su contenido.

3.2 Recomendación de prueba

Mantener como candidatas para el piloto técnico **(a) reenvío filtrado** y **(b) listener Android con allowlist de paquetes**. No construir lectura amplia de Gmail primero. La prueba debe medir: tasa de captura útil, campos disponibles por emisor, pérdida de eventos, latencia, tiempo de configuración, claridad del permiso, efecto de revocación y cantidad de contenido personal capturado. Las muestras deben ser autorizadas y anonimizadas. No priorizar una opción por los datos de S-0.1; el informe fuente identifica explícitamente ese escenario como simulado.

No es posible seleccionar responsablemente una vía definitiva solo con la documentación pública: los emisores y dispositivos prioritarios determinan la cobertura real.

4. Modelo canónico propuesto

4.1 `CaptureEnvelope`

El adaptador traduce el formato del canal a un envelope; la lógica de conciliación no conoce correo ni Android.

Campo	Tipo	Regla
`tenant_id`	UUID/string	Obligatorio; viene de la configuración autenticada del negocio, nunca del texto recibido.

Campo	Tipo	Regla
`channel`	enum	`email_forward`, `android_listener` u otro adaptador futuro.
`source_event_id`	string nullable	ID estable del mensaje/notificación si el canal lo provee.
`source_name`	string	Remitente o nombre/paquete de la app; metadato para seleccionar parser.
`title`	string nullable	Asunto o título.
`text`	string transitorio	Texto necesario para parsear; no persistir sin cifrado, acceso y retención definidos.
`received_at_utc`	timestamp	Momento de recepción por Confirma, normalizado a UTC.
`payload_fingerprint`	SHA-256	Huella derivada de campos del mensaje; no sustituye la identidad de transacción.

4.2 `PaymentNotice`

Campos normalizados mínimos: `tenant_id`, `issuer`, `amount_cop` (monto en pesos COP como entero), `currency='COP'`, `occurred_at_utc`, `reference` nullable, `operation_type`, `source_channel`, `source_event_id`, `received_at_utc`, `payload_fingerprint` y `parser_version`.

Reglas: las fechas incluyen zona horaria y se convierten a UTC; los datos ausentes permanecen nulos; el monto no se infiere del texto si el parser no puede interpretarlo sin ambigüedad. Separar el contenido crudo cifrado del pago canónico. El acceso de cajero no debe exponer configuración de captura ni credenciales.

Ejemplo esquemático (valores ficticios):

```
{
  "tenant_id": "store-001",
  "channel": "email_forward",
  "source_event_id": "mail-001",
  "source_name": "avisos@nequi.fixture.invalid",
  "received_at_utc": "2026-10-13T19:00:02Z",
  "payload_fingerprint": "sha256:<derivado>"
}
```

El envelope no incorpora el cuerpo del mensaje en este ejemplo. En producción, el tratamiento temporal del texto crudo debe definirse en función de SRS-F-14 y SRS-NFR-07: cifrado, finalidad, acceso, retención, eliminación y aviso al dueño.

5. Idempotencia y deduplicación

Se distinguen dos claves, porque repetir un mismo evento de transporte y recibir la misma transacción por dos canales son casos diferentes.

5.1 Clave de ingestión

- Con ID estable: `SHA256(tenant_id | channel | source_event_id)`.
- Sin ID estable: `SHA256(tenant_id | channel | payload_fingerprint)`.
- Repetir una misma clave no vuelve a ejecutar conciliación ni crea un segundo pago.

5.2 Identidad de pago

- Si la referencia del emisor es fiable y se valida su unicidad: `SHA256(tenant_id | issuer | normalized_reference)`.
- Sin referencia: fallback por `tenant_id | channel | payload_fingerprint`; no se fusionan automáticamente eventos entre canales distintos.

- La unicidad de referencia por emisor queda **por validar**. Si no se confirma, una posible coincidencia debe ir a revisión; una clave mal definida podría fusionar dos pagos legítimos.

Esta política permite que los fixtures de correo y Android con la misma referencia produzcan un solo pago canónico, manteniendo dos eventos de captura trazables.

6. Regla sintética de conciliación

Ventana provisional: diez minutos, alineada con SRS-RN-03 y TBD-3; configurable por negocio y sujeta a validación durante piloto.

1. Filtrar cobros pendientes por el mismo `tenant_id` y ventana temporal.
2. Si existe **una sola** candidata con monto y referencias presentes coincidentes, marcar `confirmed` y guardar un emparejamiento único.
3. Si hay más de una candidata o falta una referencia necesaria para distinguir, marcar `review`; no confirmar automáticamente.
4. Si una candidata en ventana comparte monto o referencia, pero el otro dato discrepa, marcar `doubtful` y no confirmar. Un cobro solo cercano en el tiempo, sin señal coincidente, queda `unmatched` y no se modifica.
5. Si no hay candidato en ventana, conservar el pago como `unmatched`.
6. Si el parser no reconoce o no puede interpretar el mensaje, registrar `unparsed`/`invalid`; no crear un pago.

Las consultas de candidatos siempre quedan limitadas al negocio. El cajero puede revisar el estado, pero no accede a la cuenta financiera del dueño.

7. Implementación y resultados

La vertical slice (`confirma_vertical_slice.py`) utiliza `sqlite3` en memoria, un parser de fixture sintético y dos adaptadores simples. No escribe el texto crudo en la base de datos. Se ejecuta con:

```
python3 -m unittest discover -s tests -v
python3 run_demo.py
```

Resultado de verificación: 12/12 pruebas automatizadas aprobadas; la demo produce dos observaciones de captura (correo y Android), una identidad de pago y un cobro en estado `confirmed`. Estas pruebas validan únicamente reglas sintéticas del prototipo; no validan entrevistas, preferencias, cobertura de emisores ni compatibilidad con NestJS/TypeScript.

Caso probado	Resultado esperado y observado
Pago de fixture con monto, referencia y ventana coincidentes	Una coincidencia única; `confirmed`.
Reintento del mismo ID de correo	`duplicate_event`; un pago.
Misma transacción recibida por correo y Android	Segundo evento `duplicate_transaction`; un pago canónico.
Monto discrepante dentro de ventana	`doubtful`; nunca confirmado automáticamente.
Referencia discrepante con el mismo monto	`doubtful`; nunca confirmado automáticamente.
Cobro cercano sin monto ni referencia coincidentes	`unmatched`; el cobro permanece pendiente.
Referencia ausente	`review`; el cobro sigue pendiente.
Cobro de otro negocio	Sin asociación ni cambio del cobro ajeno.
Emisor desconocido	`unparsed`; no crea pago.
Referencias diferentes, monto/fecha iguales	Dos pagos separados.
Dos cobros exactos candidatos	`review`; no elige al azar.
Coincidencia exacta más candidata sin referencia	`review`; conserva la ambigüedad.

8. Riesgos, limitaciones y decisiones abiertas

- **Formatos:** fixtures inventados; la prueba no demuestra que Nequi, Daviplata o un banco entregue esos campos o estructuras.
- **Privacidad:** el prototipo no cifra ni almacena el mensaje original. No satisface la retención/recuperación de mensajes no reconocidos de SRS-F-14; eso requiere diseño y prueba de seguridad antes de un piloto.
- **Identidad de transacción:** referencia del emisor como clave fuerte es hipótesis pendiente de validar por emisor.
- **Plataformas:** no hay pruebas reales de Android, permisos, fabricantes, Gmail ni reenviadores de terceros.
- **Seguridad y aislamiento:** las funciones simulan el filtro por `tenant_id`; no constituyen autorización de producción ni prueba de seguridad completa.
- **Persistencia:** SQLite en memoria se usa para reproducibilidad; no es decisión de motor productivo.

Próximas acciones: (1) validar con entrevistas reales el segmento, problema y alcance; S-0.1 no es evidencia de campo; (2) seleccionar emisores a partir de uso real y viabilidad, y obtener muestras autorizadas/anonimizadas; (3) ejecutar pruebas de reenvío filtrado y Android en dispositivos objetivo; (4) confirmar unicidad de referencias y comportamiento de fechas/montos; (5) acordar cifrado y retención; (6) cerrar TBD-1 mediante ADR; (7) ejecutar por separado el spike de compatibilidad de Node.js, TypeScript, NestJS y dependencias previsto para ADR-0001; y (8) portar la lógica y las pruebas a TypeScript/NestJS. **ST-01 cubre canales de captura y modelo canónico; no realiza ni sustituye el spike de compatibilidad de ADR-0001.**

9. Protocolo propuesto para la prueba con emisores

Este protocolo es una **propuesta de salida del spike**, no una decisión aprobada ni un compromiso de producto. Los mínimos se deben ajustar con el responsable y mantener coherentes con PRD/SRS antes de una prueba externa.

Dimensión	Ejecución	Criterio de salida propuesto
Cobertura y latencia	Enviar muestras autorizadas de cada emisor por cada canal en dispositivos/buzones de prueba.	Capturar al menos 95 % de las notificaciones elegibles; p95 desde recepción del canal hasta disponibilidad en pantalla ≤10 s.
Exactitud de campos	Contrastar monto, fecha, emisor y referencia con la muestra original.	100 % de campos normalizados que están presentes son exactos en el corpus; ningún dato ausente se inventa.
Idempotencia	Reentregar muestras idénticas varias veces y observar el mismo pago por ambos canales.	Cero pagos duplicados en el libro; conservar trazas de cada evento de captura.
Seguridad de conciliación	Probar coincidencia, monto/referencia discrepantes, falta de referencia y dos cobros candidatos.	Cero falsos `confirmed`; todos los ambiguos o discrepantes quedan en revisión/dudoso.
Privacidad	Enviar mensajes ajenos al allowlist/filtro y revisar almacenamiento, logs, permisos y contenido capturado.	Cero mensajes ajenos persistidos o enviados al servicio; contenido crudo cifrado y acceso/retención verificables antes de piloto.
Control del usuario	Medir configuración, revocación, reinstalación y pérdida de conectividad.	La persona entiende el alcance del permiso; al revocarlo, la captura se detiene y el estado se refleja en Confirma.

Registrar por canal/emisor: cobertura, percentiles de latencia, campos ausentes, fallos, falsos duplicados, falsas conciliaciones, tiempo de configuración, pasos de revocación y observaciones del dueño/cajero. Elegir el canal por cobertura necesaria, minimización de datos, fricción de configuración, seguridad y costo de soporte; documentar la decisión en ADR.

10. Fuentes oficiales

[1] Android Developers. “NotificationListenerService | API reference”. <https://developer.android.com/reference/android/service/notification/NotificationListenerService>. Consultado el 13-oct-2026.

[2] Android Developers. “Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS | API reference”. <https://developer.android.com/reference/android/provider/Settings#ACTION_NOTIFICATION_LISTENER_SETTINGS>. Consultado el 13-oct-2026.

[3] Google for Developers. “Choose Gmail API scopes”. <https://developers.google.com/workspace/gmail/api/auth/scopes>. Consultado el 13-oct-2026.

[4] Gmail Help. “Automatically forward Gmail messages to another account”. <https://support.google.com/mail/answer/10957?hl=en>. Consultado el 13-oct-2026.

11. Hito independiente pendiente — Compatibilidad del stack (ADR-0001)

Estado: no ejecutado; fuera del alcance de ST-01. Este informe compara canales de captura y prueba una especificación ejecutable en Python. No selecciona ni certifica versiones de Node.js, TypeScript, NestJS o dependencias.

El siguiente spike debe ejecutarse por separado, dentro del repositorio objetivo y con el entorno de CI previsto. Su resultado se documentará en ADR-0001, registrando versiones exactas y evidencia reproducible; las versiones a ensayar deben elegirse según soporte vigente al momento de ejecución, no inferirse de S-0.1.

Matriz mínima de comprobación propuesta:

1. Fijar una versión candidata de Node.js y combinaciones compatibles de TypeScript, NestJS y paquetes críticos; documentar sistema operativo y gestor de paquetes.
2. Instalar desde cero con archivo de lock; comprobar `build`, comprobación estática, pruebas unitarias y arranque de la aplicación.
3. Verificar compatibilidad de adaptadores de persistencia y migraciones, configuración por entorno, registro y manejo de errores.
4. Probar construcción reproducible de imagen/contenedor y ejecución en CI, si el despliegue objetivo los requiere.
5. Registrar incompatibilidades, criterios de soporte/actualización y decisión final de versiones en ADR-0001.

Criterio de salida sugerido, pendiente de aprobación: combinaciones fijadas y reproducibles; instalación limpia; compilación, pruebas y CI exitosas; dependencias críticas inventariadas; estrategia de actualización y soporte aceptada por el responsable.

La vertical slice Python puede servir de referencia para portar contrato y reglas, pero no sustituye estas pruebas. Una vez aprobado el stack, portar las doce pruebas automatizadas y los fixtures a TypeScript/NestJS antes de considerar la lógica parte del backend objetivo.